YAMLess — pos-elera-klog-integrator

YAMLess: Generating Azure Infrastructure from Code

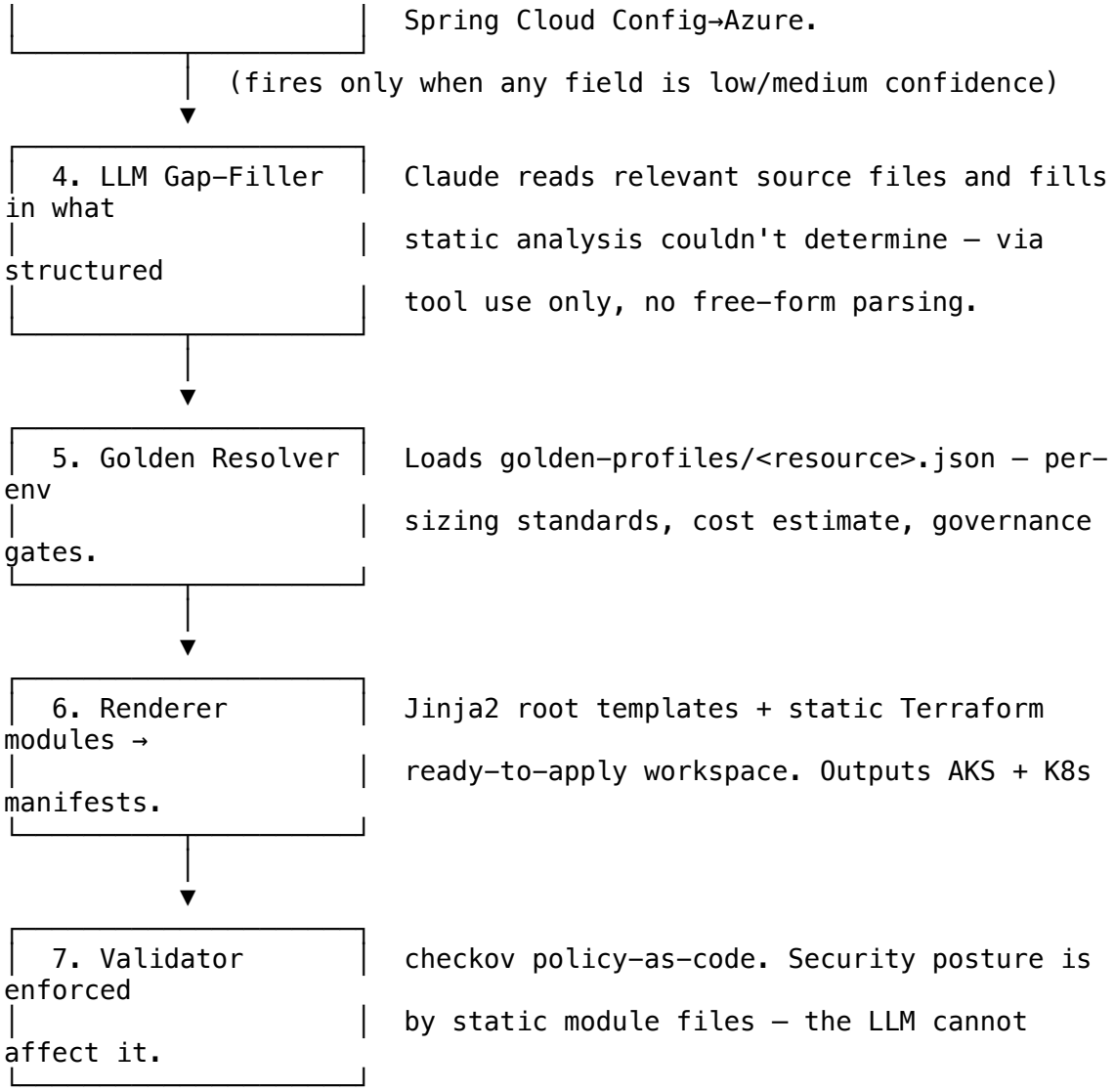
Case Study — krogertechnology/pos-elera-klog-integrator

What is YAMLess?

YAMLess is an **Infrastructure from Code (IFC)** tool. Given a code repository, it produces a complete, policy-validated Terraform workspace — with no configuration files, no schema to fill in, and no IaC expertise required from the developer.

The pipeline has seven stages:

```
Repo on disk
    │
    ▼
┌──────────────────────┐
│  1. Static Analyzer  │  Reads Dockerfile, pom.xml, .env files, Helm values,
│                      │  Spring Boot YAML, source code (Java @Value, env refs).
│                      │  Produces a typed AppProfile with confidence scores.
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│  2. Runtime Enricher │  Java-specific: spring-cloud-starter-config detection,
│                      │  management.server.port, JMX port from JAVA_OPTS.
│                      │  CLOUD_CONFIG_URL env var also signals Spring Cloud Config.
└──────────┬───────────┘
           │
           ▼
┌──────────────────────┐
│  3. Migration Risk   │  Static table (source tech × target cloud) → warnings.
│     Evaluator        │  Entries: RabbitMQ→ServiceBus, MongoDB→CosmosDB,
```



Applying YAMLess to pos-elera-klog-integrator

pos-elera-klog-integrator is a Kroger Technology Java Spring Boot microservice that integrates with the ELERA POS platform. It routes POS events via IBM MQ / AMQP and Azure Camel Service Bus, persists data to blob storage, exposes metrics via a JMX Prometheus exporter (port 12345), and runs a Helm-managed cron job for periodic reconciliation.

Command run

```
yamless generate /Users/zafar/pos-elera-klog-integrator \
  --no-llm \
  --target aks \
  --out-dir ~/yamless/outputs/pos-elera-klog-integrator-aks-infra-v1
```

Profile detected

Field	Detected value	Confidence	Notes
Runtime	`java`	high	`FROM .../java-openjre:25-latest`
Runtime version	`25`	high	
Port	`null`	—	`EXPOSE $SERVER_PORT` — variable reference, not literal
Target port	`8080`	—	Fallback from Helm `port: 8080`
Shape	`http-service`	high	`spring-boot-starter-web` detected
Health check path	`/actuator/health`	—	`spring-boot-starter-actuator` detected
Memory limit	`3Gi`	—	Helm `memory.limits: 3Gi`
Memory request	`1.6Gi`	—	Helm `memory.requests: 1.6Gi`
Database	`null`	—	No DB client artifact; `MSSQL_SA_PASSWORD` in secrets (likely tooling)
Cache	`true`	high	`jedis` Redis client in pom.xml
Queue	`true`	high	`com.ibm.mq.allclient`, `mq-jms-spring-boot-starter`, `spring-boot-starter-amqp`, `camel-azure-servicebus`
Kafka	`false`	—	
Storage	`true`	high	`pos-elera-blob-persistence`
Secrets	1 detected (`MSSQL_SA_PASSWORD`)	high	Low count — secrets likely injected at runtime
Min replicas	`1`	—	
Max replicas	`10`	—	
Ingress	`external: true`, port `8080`	high	
Scale trigger	`http`	high	
additional_ports	`[]`	—	`EXPOSE $PROMETHEUS_PORT` uses variable — not matched by literal regex
spring_cloud_config	`false`	—	No `spring-cloud-starter-config`, no `CLOUD_CONFIG_URL`

Field	Detected value	Confidence	Notes
workers	[]	—	Helm cron job not detected — shape_detector looks for dep packages, not Helm

Output generated

```
pos-elera-klog-integrator-aks-infra-v1/
├── profile.json
├── pos-elera-klog-integrator-yamless-report.md
├── pos-elera-klog-integrator-cost-report.html
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── providers.tf
│   ├── backend.tf
│   ├── governance.tf
│   └── modules/
│       ├── aks_cluster/
│       ├── aks_workload_identity/
│       ├── keyvault/
│       ├── redis/
│       ├── servicebus/
│       ├── storage/
│       ├── vnet/
│       └── (postgres, mysql, cosmos, event_hubs — generated but
unused)
└── k8s/
    ├── namespace.yaml
    ├── service-account.yaml
    ├── secret-provider-class.yaml
    ├── deployment.yaml
    ├── service.yaml
    ├── ingress.yaml
    ├── hpa.yaml
    └── network-policy.yaml
```

38 .tf files + 8 K8s manifests written.

Warnings emitted

#	Warning	Source
1	RabbitMQ → Service Bus: AMQP 0-9-1 vs 1.0 protocol gap; direct reply-to unsupported	#37 Migration Risk Evaluator

```
  ! RabbitMQ → Azure Service Bus: AMQP 0-9-1 vs AMQP 1.0 protocol gap.
    Direct reply-to (amq.rabbitmq.reply-to /
RabbitTemplate.convertSendAndReceive)
    is not supported on Service Bus — migrate to correlation-ID reply
patterns before go-live.
```

Cost estimate

Environment	Monthly (low–high)	Annual
Dev	$170 – $240	$2,040 – $2,880
Staging	$357 – $427	$4,284 – $5,124
Production	$1,047 – $1,117	$12,564 – $13,404

Detection Analysis

What YAMLess got right

Finding	Method
Runtime: Java 25	`FROM` base image parse
Health check: `/actuator/health`	`spring-boot-starter-actuator` dep → shape_detector
Memory: 3Gi limit / 1.6Gi request	`scan_helm_resources()` reads Helm values.yaml
Queue detected (`com.ibm.mq.allclient`, `spring-boot-starter-amqp`)	`analyzer/dependencies.py` pom.xml artifact scan
Storage detected (`pos-elera-blob-persistence`)	custom package → storage flag
Cache detected (`jedis`)	Redis client artifact → cache flag
AMQP migration risk warning	Migration Risk Evaluator: `(rabbitmq, azure)` table entry
`target_port: 8080`	Helm `port: 8080` fallback when EXPOSE is a variable

Known gaps

Gap	Root cause	Potential fix
`port: null`	`EXPOSE $SERVER_PORT` uses an ENV variable reference; `_detect_ports()` only matches literal integers	Resolve `ENV SERVER_PORT=...` declara and substitute before port detection
`additional_ports: []` (Prometheus 12345 missed)	Same root cause: `EXPOSE $PROMETHEUS_PORT` uses an ENV var	Same fix as above; alternatively parse JM agent flag (`javaagent:./jmx.jar=$PROMETHEUS_`
`workers: []` (Helm cron job not detected)	Shape detector infers workers from dependency	Add Helm `cronjobs:` block parsing to `scan_helm_resources()`

Gap	Root cause	Potential fix
	packages (e.g. `celery`, `rq`), not from Helm chart definitions	
`database: null` (despite `MSSQL_SA_PASSWORD`)	No SQL Server JDBC driver or Spring Data artifact in pom.xml — likely a test/tooling variable	Likely correct; could add `MSSQL_SA_PAS` as a heuristic signal

Infrastructure Generated

The rendered output covers all detected dependencies:

Resource	Terraform module	K8s resource
AKS Cluster	`modules/aks_cluster/`	—
Workload Identity (UAMI)	`modules/aks_workload_identity/`	`service-account.yaml`
Key Vault	`modules/keyvault/`	`secret-provider-class.yaml`
Redis Cache (P1 prod)	`modules/redis/`	—
Blob Storage	`modules/storage/`	—
Azure Service Bus (Standard)	`modules/servicebus/`	—
VNet	`modules/vnet/`	—

Kubernetes manifests include topology spread constraints (zone + hostname), HPA (1–10 replicas), network policy, and Workload Identity service account annotation.

Improvement Roadmap

Priority	Change	Status
High	Resolve ENV var references before port EXPOSE matching	Open
Medium	Parse Helm `cronjobs:` block to populate `workers` list	Open
Medium	Treat `MSSQL_SA_PASSWORD`	Open

Priority	Change	Status
	secret as SQL Server signal	

Summary

YAMLess v1 against `pos-elera-klog-integrator` correctly identifies the Java runtime, Spring Boot health check path, Helm-sourced memory limits, all three backing service dependencies (Redis, Service Bus, Blob Storage), and fires the AMQP migration risk warning. The main gap is Dockerfile port detection: both `EXPOSE $SERVER_PORT` and `EXPOSE $PROMETHEUS_PORT` use ENV variable references that the current literal-integer regex cannot match. Port 8080 is recovered via Helm fallback; the Prometheus sidecar port (12345) is missed. The Helm-defined cron job is also not surfaced as a `WorkerProfile` — a future enhancement to `scan_helm_resources()`.

Generated by YAMLess — https://github.com/zafar-ahmed-kr/YAMLess Date: 2026-11-11